

OPEN SOURCE ENGINE BLUEPRINT

Building a Disciplined Agent Execution Layer

Beyond vibe coding: using markdown-driven contracts to run
predictable, self-correcting AI engineering workflows.

PROMPT-DRIVEN DEVELOPMENT

EXECUTABLE SPECIFICATIONS

AUDITABLE RUNS

Where Does This Fit in an Agent Factory?

To systematically clear a software backlog with AI, you need a decoupled architecture.

LAYER 1

Backlog Runner

Breaks functional product features into discrete, claimable work items for autonomous execution.

LAYER 2

Methodology Runner

Orchestrates multi-phase engineering methods: requirements, architecture, design, implementation, validation.

LAYER 3

Prompt-Runner

The low-level execution engine. It processes prompt pairs, manages agent sessions, runs validators, and logs immutable evidence.

OPERATING MODEL

Parallel Agents Require Isolated Workspaces

Production code paths

Completed deliverables land in real project-visible paths such as `src/module.py`, `tests/test_module.py`, or `docs/design.yaml`.

CLEAN DIFF

REVIEWABLE FILES

Scratchpad evidence paths

Temporary files, rendered prompt inputs, LLM outputs, raw logs, metrics, and summaries stay isolated inside `.run-files`.

AUDIT TRAIL

DEBUG CONTEXT

The run directory is an isolated worktree. Agents can operate without corrupting active project state or polluting version control with execution debris.

MARKDOWN CONTRACT

Specs as Code: The Markdown Contract

```
## Prompt: Implement parser
```

```
### Required Files  
docs/request.md
```

```
### Generation Prompt  
Build the behavior.
```

```
### Validation Prompt  
Audit the result and emit a verdict.
```

Document structure is execution logic

Instead of ad-hoc chat sessions, workflows are declared as structured markdown contracts.

Hard boundaries

Prompt headings define sequence. Required files are checked before network calls. Generation and validation roles are explicitly separated.

EXECUTION LOOP

The Generate, Audit, and Commit Cycle

STEP 1

Render

Combines placeholders, file includes, and system preludes into exact backend inputs.

STEP 2

Generate

Calls the code generator session and modifies the isolated workspace.

STEP 3

Validate

Runs local checks such as tests, linters, type checks, or schema validators.

STEP 4

Judge

A separate AI validation agent audits the artifact and issues a formal verdict.

STEP 5

Commit evidence

Persists metrics, logs, summaries, prompt inputs, and outputs to the audit trail.

This loop is the antidote to vibe coding: every generated artifact is tied back to an executable specification and a validation record.

Managing State to Prevent Token Waste

The failure mode

Long-running agent pipelines hit network drops, rate limits, validation halts, or local test failures. Restarting from scratch burns time and API budget.

The recovery model

Prompt-runner caches execution state and generated outputs to disk. Resume can pick up at the interrupted point without replaying completed LLM calls.

PERSISTED STATE	WHY IT MATTERS
Rendered prompt inputs	Exact instructions are reproducible after the run
Generated artifacts	Completed steps are reused instead of regenerated
Verdicts and summaries	Later steps know which work is already approved

HARD RULES VS. SOFT LOGIC

Anchoring AI Behavior with Deterministic Checks

Generate code

The agent writes or updates files inside the run worktree.

Run local validator

Tests, type checks, schema linters, or custom scripts run before the LLM judge call.

Feed objective evidence

The judge receives concrete validation logs alongside the generated artifact.

Exit code 0

Tests pass. Success logs are fed to the AI judge to verify functional intent.

Exit code 1

Tests fail. Failure logs drive automated revision instead of a loose review.

Exit code above 1

The validator itself failed. The run halts so tooling can be fixed.

Decoupling Roles to Optimize Unit Economics

One premium model for every task is an expensive default. The engine lets the builder and auditor take different routes.

The Builder

Bulk code generation can use faster, high-throughput, or specialized coding models where the task is mostly implementation work.

GENERATION

The Auditor

Validation prompts and selector choices can use premium reasoning models where correctness and judgment matter most.

JUDGING

SELECTION

Role-specific routing improves aggregate cost without collapsing the quality bar for the final decision.

Running Parallel Experiments with Variant Forks

```
<svg viewBox="0 0 980 320" style="width:100%; height:320px; background:var(--  
surface); border:1px solid var(--border); border-radius:8px;"> <rect x="60" y="122"  
width="170" height="76" rx="10" fill="#101522" stroke="#202938"/> <text x="92"  
y="155" fill="#f8fafc" font-size="19" font-weight="800">One spec</text> <text x="92"  
y="181" fill="#a4adba" font-size="14">same contract</text> <path d="M248 160  
C334 82, 400 70, 486 92" fill="none" stroke="#687385" stroke-width="2"/> <path  
d="M248 160 C334 238, 400 250, 486 228" fill="none" stroke="#687385" stroke-  
width="2"/> <rect x="508" y="52" width="190" height="86" rx="10" fill="#111827"  
stroke="#38bdf866"/> <text x="545" y="89" fill="#38bdf8" font-size="19" font-  
weight="800">Variant A</text> <text x="545" y="116" fill="#a4adba" font-  
size="14">isolated run dir</text> <rect x="508" y="184" width="190" height="86"
```

SELECTION MECHANICS

How the Engine Selects the Winning Code Path

01

A passing validation verdict always beats a non-passing candidate.

02

Fewer revision iterations win because first-pass correctness matters.

03

Lowest wall-clock execution time wins when quality ties.

04

Total token consumption acts as the final tie-breaker.

Selector Judge

After candidates run, a dedicated selector evaluates scorecards and outcomes. This keeps selection disciplined instead of relying on a loose preference call.

Variant A

Pass, 2 revisions, 41k tokens

Variant B

Pass, 1 revision, 37k tokens

Winner

Variant B

PRODUCTION-GRADE ROBUSTNESS

Built for Stability, Not Demos

Higher-level backlog orchestrators only work if the execution layer is boring, strict, and repairable.

255+ tests

Regression coverage for parser behavior, iteration loops, variant forks, and CLI config discovery.

Typed errors

Parser failures use explicit error IDs and line references so malformed specs are repairable.

Role isolation

Generation, validation, selection, and local validators each have a clear place in the lifecycle.

Evidence first

Every meaningful call leaves logs, metrics, outputs, and summaries for later inspection.

E-NO-GENERATION

E-BAD-SECTION-ORDER

E-DUPLICATE-SECTION

E-BAD-PATH-ENTRY

E-UNKNOWN-SUBSECTION

E-NO-VARIANTS

SUMMARY AND NEXT

Summary: Markdown In. Auditable Code Out.

Core takeaway

Prompt-runner transforms fragile prompt strings into a version-controlled, self-correcting execution layer.

Open source posture

The tool is transparent, reproducible, and designed for professional engineering pipelines.

Coming next

At scale, automated agent work needs cost governance and behavioral insight across active runs.

Next topic: the companion reporting and observability tool for real-time token tracking and tool-call auditing.